

- Process to communicate with supply chain partners to identify, report, and correct system and component flaws in a timely manner.
- Protection from code threats originating from an unauthorized or malicious source in the supply chain. These protections should be applied throughout the supply chain.
- Monitoring for supply chain system and component security and threat intelligence alerts and advisories from supply chain partners. Effective management takes appropriate actions in response. Monitoring processes should include monitoring that lessons learned from prior compromises are appropriately acted upon. For example, if malicious code implanted during software development leads to a compromise, effective controls are implemented to further ensure code legitimacy in the future. It is important for management to consider
 - Correlation of available threat intelligence information (e.g., internally produced and from supply chain partners) to identify potential threats or vulnerabilities requiring mitigation.
 - Employment of enhanced monitoring over activities performed by higher-risk personnel (e.g., users with elevated authority or privileges).
- Determining periodically whether any counterfeit or shadow IT systems and components exist in the supply chain. Typically, management addresses this issue through contractual provisions with supply chain partners regarding their mitigation program for shadow IT risk. If any are discovered, management should assess the impact of removal, then remove any suspect systems or components from the entity's infrastructure as appropriate.¹⁶⁵
- **Maintenance** includes performing system and component updates and replacements and may be performed by a third party. When performed by a third party, maintenance becomes part of the supply chain and SCRM principles should be applied. Management should perform the following when implementing maintenance processes throughout the supply chain:
 - Define agreements that identify roles, responsibilities, and practices that may be used for maintenance activities, especially when third parties are responsible for maintaining the entity's systems or components.
 - Monitor for unauthorized modification or removal of the entity's systems or components (e.g., use of counterfeit systems, counterfeit components, and malware) in the supply chain. Agreements with supply chain partners should address monitoring and communication to affected parties throughout the supply chain.
 - Monitor systems and components for planning for EOL to prepare for replacement or upgrade to the systems and components. Effective management considers any potential availability issues in the supply chain for systems and components or has agreements to continue support for legacy systems or components until a change can be made without significant disruption to operations.
- **System or component EOL and disposal processes** outline the steps personnel should take when components are nearing EOL and need to be decommissioned and removed from the entity's infrastructure. Standard EOL processes should be followed. Additional processes related to supply chain activities may involve the following:

¹⁶⁵ For more information on shadow IT and IT asset management, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" booklet.

- Inclusion of data disposal processes to allow entity management to mitigate the risk that entity data maintained by supply chain partners may be retained or compromised.
- Consideration of supply chain partner interdependencies associated with the removal of a system or component from the entity's environment, as well as throughout the supply chain. For example, the removal of a supply chain partner's system or component that may disrupt the entity's ability to provide continuous service to customers.
- Sanitization or destruction of information from supply chain partner systems, components, and media before disposal or release for reuse. Agreements should address this for all locations and providers in the supply chain. Off-boarding procedures should be addressed in agreements to maintain data confidentiality, integrity, and availability until the relationship with that supply chain partner is officially terminated.
- Application of data protection controls (e.g., tracking chain of custody) throughout off-boarding processes for an entity's data and for the data received from supply chain partners.

Additionally, as a part of SCRM, entity management should consider interconnectivity risks throughout the supply chain. What may initially be assessed as a low-risk activity or asset may be considered a high-risk activity or asset based on the interconnectivity or reliance on that system or component. For example, a system or component (e.g., HVAC, smart TVs, and IoT devices) may not appear high-risk, yet the system or component may be connected to critical systems and components throughout the supply chain, and subject to remote access risks. Therefore, management should have a process to validate the effectiveness of the security of systems and components throughout the supply chain when performing development, acquisition, and maintenance activities, and make appropriate adjustments to risk assessments to account for interconnectivity risk.

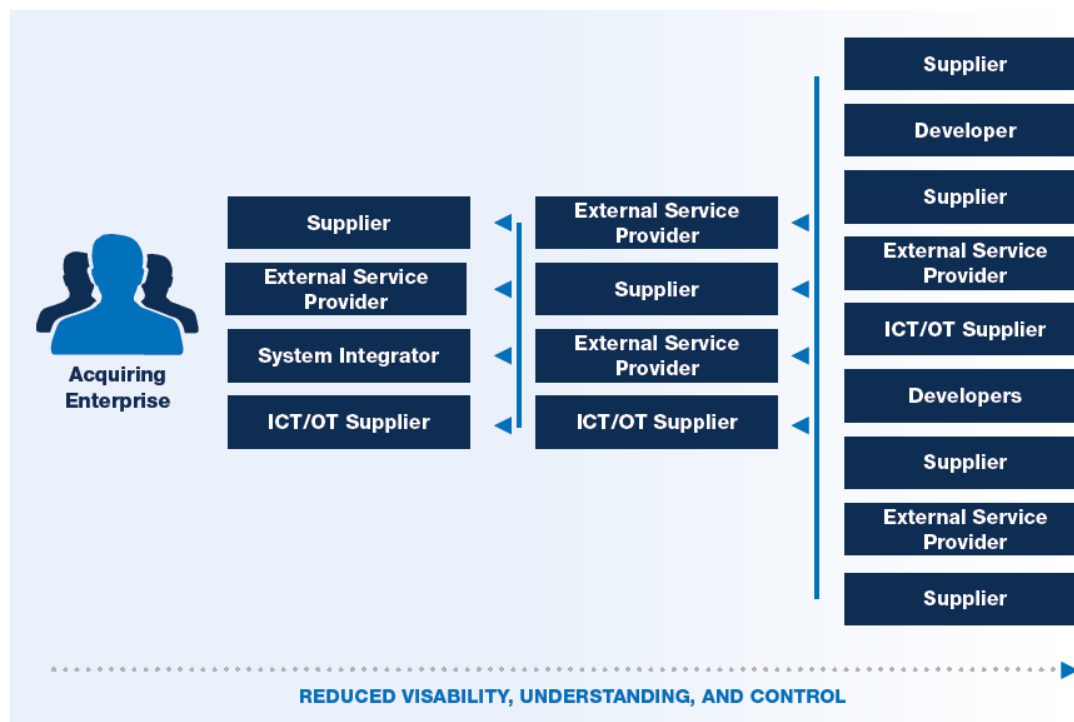
To effectively manage supply chain risks, management should have a clear understanding of interconnectivity in the entity's supply chain. To facilitate this understanding, management should consider using available information, such as that provided by third-party user groups and associations,¹⁶⁶ which can augment ongoing monitoring and due diligence, threat intelligence, and security throughout the supply chain. A vulnerability to one system, component, or supply chain partner may pose a vulnerability to the entire supply chain. Figure 5 depicts diverse supply chain relationships that affect an entity's visibility and control of the supply chain. Entities depend on the supply chain to provide a variety of products and services to enable the enterprise to achieve its strategic and operational objectives. As NIST states, acquirers often lack visibility and understanding of how acquired technology is developed, integrated, and deployed and how the services that they acquire are delivered. Additionally, acquirers with inadequate or absent SCRM processes and practices may experience increased exposure to cybersecurity risks throughout the supply chain. The level of exposure to cybersecurity risks throughout the supply chain depends largely on the relationship between the products and services provided and the

¹⁶⁶ Associations may include standards-setting organizations, such as the National Automated Clearinghouse Association ([Nacha](#)), and [PCI Security Standards Council](#) (for the payment card industry) and information-sharing organizations (e.g., [Financial Services Information Sharing and Analysis Center \[FS-ISAC\]](#), and other information-sharing and analysis centers [[ISAC](#)]).

criticality of the missions, business processes, and systems they support.¹⁶⁷ Entities have a variety of relationships with their suppliers, developers, system integrators, external system service providers, and other information communication and technology/operational technology-related¹⁶⁸ service providers.¹⁶⁹

Effective management considers risk management factors over the entire life cycle of a third-party relationship, including planning, due diligence, contract negotiation, ongoing monitoring, and termination. For more information on these factors, refer to the *FFIEC IT Handbook's* “[Outsourcing Technology Services](#)” and “[Management](#)” booklets. Additionally, for more information on supply chain relationships, refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#).

Figure 5: Supply Chain Relationships



¹⁶⁷ Refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#), for more information.

¹⁶⁸ NIST Glossary defines “[information and communication technology](#)” as “encompassing the capture, storage, retrieval, processing, display, representation, presentation, organization, management, security, transfer, and interchange of data and information.” It defines “[operational technology](#)” as “programmable systems or devices that interact with the physical environment (or manage devices that interact with the physical environment).”

¹⁶⁹ Refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#).

IV.Q.2 Software Bill of Material

An SBOM is a formal record containing the details and supply chain relationships of various components used in building software.¹⁷⁰ “A primary purpose of an SBOM is to identify components and their relationships to one another, using some combination of baseline component information.”¹⁷¹ There are benefits to supply chain partners including developers, purchasers, and operators of software. An SBOM can help developers of systems and components be aware of any software they might use in development of entity solutions. For purchasers, an SBOM helps identify software supporting a product potentially subject to compatibility issues. An SBOM enables an entity and its operators to determine in a relatively short time whether the entity is affected by a vulnerability and where in the supply chain the entity may have been affected¹⁷² because management can match vulnerabilities to elements in the SBOM.

Management may use an SBOM to identify what systems and components fall outside the entity’s risk tolerance thresholds (e.g., has a significant vulnerability, inappropriate use of open-source components, or is linked to an untrusted provider). An SBOM may help with the management of OEM licensing and compliance and enable management to quickly identify interdependencies and other supply chain risks. An SBOM gives management visibility into systems and components used in the entity’s infrastructure, including open-source systems and components and IoT products.¹⁷³ This visibility allows management to perform activities such as the following:

- Determine whether any of the software or components used in the entity’s infrastructure are subject to specific vulnerabilities or potential risks associated with publicized breaches, as determined through entity threat intelligence processes.
- Uniquely identify systems and components used by customers and authorized entities.
- Trace users and the use of licenses, allowing management to determine whether access control information is accurate and whether its use is appropriate.

Tools used to read an SBOM should “uniquely identify individual components in a standard format,”¹⁷⁴ so that all references are uniform throughout the supply chain. Additionally, an SBOM should be updated whenever software elements are changed or software is updated.

¹⁷⁰ Refer to [NIST Glossary](#).

¹⁷¹ Refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#).

¹⁷² Refer to National Telecommunications and Information Administration (NTIA), [Framing Software Component Transparency: Establishing a Common Software Bill of Materials \(SBOM\)](#) and “[SBOM Myths vs. Facts](#).”

¹⁷³ For more information, refer to NIST IR 8425, [Profile of the IoT Core Baseline for Consumer IoT Products](#), and NIST IR 8259A, [IoT Device Cybersecurity Capability Core Baseline](#).

¹⁷⁴ Refer to NTIA’s [Framing Software Component Transparency: Establishing a Common Software Bill of Materials \(SBOM\)](#) and “[SBOM at a Glance](#).”

SBOMs typically do not identify intellectual property, patents, algorithms, or code; if they do, management should implement appropriate security for this proprietary information.

NIST states that as SBOMs mature, management should help ensure that they do not deprioritize existing SCRM capabilities (e.g., vulnerability management practices and vendor risk assessments) under the mistaken assumption that SBOMs replace these activities. Accurate SBOMs are meant to provide improved transparency, which is complementary to an entity's risk management practices, not a substitute for them. "Entities that are unable to appropriately ingest (i.e., recognize and understand), analyze, and act on the data that SBOMs provide likely will not improve their overall SCRM posture."¹⁷⁵

IV.Q.3 Enterprise Risk Management and Supply Chain Risks

SCRM touches on risk management activities across the entity and should be incorporated into enterprise and technology risk management activities. A few examples of security elements tied to SCRM include the following:

- **Confidentiality:** Supply chain partners may receive sensitive entity and customer information.
- **Integrity:** Systems or components may not operate as designed, leading to a data integrity issue, or third parties may intentionally or unintentionally change entity, partner, customer, or consumer data.
- **Availability and resilience:** Systems, components, or services may not be available or recoverable when needed.

According to NIST, "Managing supply chain risk is a complex, multifaceted undertaking that requires a coordinated effort across an organization to build trust relationships and communicate with internal and external stakeholders."¹⁷⁶ Supply chain activities occur throughout the SDLC; therefore, SCRM should be embedded throughout the SDLC.¹⁷⁷ Additionally, when applicable, management should incorporate third parties into business continuity and resilience activities throughout the supply chain.

¹⁷⁵ Refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#).

¹⁷⁶ Refer to NIST 800-53, rev. 5, [Security and Privacy Controls for Information Systems and Operations](#).

¹⁷⁷ Refer to NIST SP 800-161r1, [Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations](#).

V DEVELOPMENT

Action Summary

Implementation of sound processes for the development of systems and components supporting the entity's business needs and operations is important.

Examiners should review the following:

- Documentation of training in secure design and coding techniques for those entity personnel who are responsible for development.
- Development standards, including procedures for managing changes and a back-out plan.
- Evidence of communication throughout the supply chain regarding identification of critical systems and components, potential threats and vulnerabilities, configurations, and responsibilities for selection and maintenance of system and network components.
- Entity coding standards and list of entity-approved software development languages.
- Documentation of security certification for completed systems and component-related code.
- Development designs used to validate security and functionality throughout the supply chain.
- Risk assessment documentation related to the use of or switch to a development operations (DevOps) approach, or extending to a development security operations (DevSecOps) approach, and effectiveness of controls.

Development is the systematic application of knowledge toward the production of useful materials, devices, and systems. In this context, it involves the processes of defining, designing, developing, testing, and implementing systems or components. Development includes validation and demonstration of a chosen technology, use of test and production environments, improvement of developed prototypes, integration into systems and subsystems, and hardware builds. Understanding development concepts is important to all entities whether development is performed in-house or by a third party on the entity's behalf. Additionally, development can be a complex topic, which requires the entity's board, senior management, and business line management to understand risks and communicate effectively. For entities that develop or modify their own systems and software, effective management requires training in secure design and coding techniques of those responsible for development. If formal standards are not in place and development processes are not controlled, the following can occur, affecting confidentiality, integrity, availability, and resilience:

- Corruption (e.g., malware or sabotage) of the development process.
- Unintentional creation of vulnerabilities (e.g., backdoors or open ports).
- Omission or errors in process or code (e.g., intended processes that were forgotten or calculations that are incorrect) resulting in production issues (e.g., disruptions or delays).

Entity personnel should maintain and follow documented development policies, standards, and procedures that result in systems and components that meet the entity's requirements for confidentiality, integrity, availability, and resilience. Standards and procedures should identify approved development tools and tool configurations in the development process or in a specific development methodology.

When engaged in development, management may consider using design and coding techniques, appropriate tools (e.g., computer-aided design), and prototypes. When development personnel use design and coding techniques, they should appropriately manage risks (e.g., eliminating inaccurate techniques or outdated tools and prototypes). Prototypes may help illustrate a system's or component's functionality to stakeholders and help management identify, evaluate, and highlight design risks not apparent during the impact analysis or before production. Then the design risks may be resolved more efficiently. There are alternatives to prototyping, such as defining and building a system or component with only enough features to accomplish the essential user objectives of the system.

As part of the SDLC, effective management works toward developing a trustworthy system. From a security perspective, a trustworthy system¹⁷⁸ meets specific security requirements and other critical requirements defined and set by entity management. To help develop a trustworthy system, management should employ secure program coding practices. Secure coding practices help developers minimize the potential for known source code-related vulnerabilities in the development phase.

Examples of secure coding practices include the following:¹⁷⁹

- Validate all inputs and validate and properly encode all outputs.
- Avoid using unsafe functions.
- Detect and correct errors effectively.
- Provide logging and tracing capabilities.
- Consider use of automated features that encourage secure coding practices in development environments, when appropriate.
- Establish procedures for manually applying secure coding practices when automated methods are insufficient or unavailable.
- Use tools (e.g., linters¹⁸⁰ or formatters) to analyze code, identify common mistakes and bad coding style, and standardize the style and formatting of the source code.
- Check for common system, code, and development environment vulnerabilities.
- Assign developers to review code to complement (not replace) independent code review and verify compliance with security requirements.

¹⁷⁸ Refer to NIST SP 800-160, vol. 1, rev. 1, [Engineering Trustworthy Secure Systems](#).

¹⁷⁹ Refer to NIST SP 800-218, [Secure Software Development Framework \(SSDF\) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities](#).

¹⁸⁰ For purposes of this booklet, a "linter" is a type of tool used to perform code analysis to flag issues, such as programming errors, bugs, and stylistic errors.

- Identify and correct vulnerabilities before release of software to production to prevent exploitation.

Developers should communicate throughout the supply chain to ensure that partners are aware of potential threats and vulnerabilities when developing, testing, and maintaining systems and components. Additionally, the individuals responsible for selecting system and network components should incorporate supply chain processes when choosing such components. Developer training should include secure coding and the use of tools to find vulnerabilities in systems and components across the supply chain. For more information, refer to the “[Supply Chain Considerations](#)” section of this booklet.

V.A Development Standards and Controls

Management should establish development standards, including procedures for controlling changes during the development process that address the following:

- **System controls**, including a system’s or component’s functionality, security, automated control features, and coding standards.
- **Quality management** (also referred to as quality assurance and quality control [QA/QC]), including the development and implementation of a system or component so it meets predefined specifications and management’s expectations.
- **Release management**, including migration of the final release of systems or components from a development or development testing environment into the production environment for deployment. This may include multiple steps. A release can be completed manually, or the release process could be automated. Established rollback instructions, or back-out plans, should be developed and available in the event of unforeseen deployment problems.
- **Documentation** to facilitate stakeholders’ understanding of the following:
 - Development process.
 - Error correction and program modification processes.
 - System or component business functionality.
- **Reporting**, including processes for generating and managing reports (e.g., types of reports, timing, and stakeholders who should receive them) during development to help management make important budgeting and other data-driven decisions. Reports could include project updates, technical issue logs, and change management logs.

Development standards should articulate the entity’s minimum development requirements. For example, development standards may address the selection of programming styles, languages, and tools; layout or format of scripted code; naming conventions; and program library requirements. Effective management communicates in policy its reasons for using specific programming styles and languages on a project or service in project documentation. This is done to identify decisions made by management so developers can understand the reasons certain programming styles and languages were used and whether they remain appropriate for current functionality and stakeholder needs. Examples are included in table 3.

Table 3: Descriptions and Reasons for Using Programming Styles and Example Languages

Programming style	Description	Why	Example languages ¹⁸¹
Compiled	Source code converted into machine language before the code can operate	- Useful when resources are limited and execution speed (for transactions) is essential. - Less portable than hybrid or interpreted languages due to the need to recompile for each new system.	C, C++, GO, Swift, Objective-C, Visual Basic
Hybrid (compiled and interpreted)	Uses a combination of compiled and interpreted languages programming style	- Portable and slower than compiled. - Requires more resources (e.g., CPU and random access memory [RAM]) than compiled, but less than interpreted language.	Java, C#, Clojure, Scala, Spark, Prolog
Interpreted ¹⁸²	Lines of code that can operate without first being compiled	- Less complex than compiled or hybrid programming styles, but requires more resources (e.g., CPU and RAM). - Easier to find trained developers, as less technical knowledge is needed to implement the code.	Python, JavaScript, PowerShell, PHP, Structured Query Language (SQL), Ruby, Perl, Lua, Delphi
Assembler	Converts assembly instructions into machine language	- Useful when you need to minimize resources (e.g., CPU and RAM) needed to implement the code. - Often used for firmware.	Assembly
Markup language	Language with rules and instructions accompanying the code, which facilitates use by humans and programs	Useful when text needs to be formatted in a specific way (e.g., font, color, and size).	HTML, XML, SGML
Legacy	Older programming languages that potentially are unsupported, general use is declining	- Personnel with knowledge of code development and use is declining, making it difficult to maintain and upgrade. - Concerns related to programming style in sunset phase of SDLC.	FORTRAN, COBOL, APL, ALGOL, LISP, PASCAL, Assembly, REXX, JCL

The use of naming conventions¹⁸³ for a software program’s modules and any related subroutines, databases, or programs that interact with an application help a team of developers work more efficiently together. For example, naming convention use promotes consistency so programmers can link subroutines into a unified program more efficiently. Naming conventions also facilitate understanding so IT staff not involved in the original development can later modify systems

¹⁸¹ Languages may be applicable in several different language styles; these are just examples.

¹⁸² An interpreted programming style refers to processing a script or other program expressions line by line and in accordance with the language requirements, without first compiling a program into machine instructions.

¹⁸³ NIST defines a “[naming convention](#)” as “a collection of rules, which when applied to data, results in a set of data elements named in a logical and standardized way. These names inform the user about the contents of the data value domain, and the usage of the data element, in a concise manner.”

efficiently. Finally, use of naming conventions often facilitates software code reuse and repurposing.

To establish development standards and controls, prudent practices include thorough development testing and validation of systems and component-related code. Effective management keeps completed systems and component-related code that have passed security certification in program libraries as discussed in the “[Additional Control Considerations in Change Management](#)” section of this booklet.

Due to supply chain risk in development, for both in-house and supply chain partners, it is important to consider using the following standards and controls when developing information systems and components:

- Design security controls to be difficult to disable (e.g., tamper-proofing techniques), and, if they are disabled, trigger notification methods such as audit trails, tamper evidence, or alarms.
- Design delivery mechanisms (e.g., downloads for software) to avoid unnecessary exposure or access to the supply chain and systems or components traversing the supply chain.
- Design relevant validation mechanisms to be used during implementation and operation.

Developer configuration management is critical for reducing cybersecurity risks, both in-house and throughout the supply chain. Developers should manage the configurations of the development environments in which they are working. They should create configuration guides for the systems and components they are developing for distribution with the system or component. By implementing configuration management standards and controls, developers reduce the occurrence and likelihood of flaws while increasing accountability and ownership for the changes. Developer configuration management should be performed by developers, including any third parties involved in the process.

If management decides, based on assessments of risk throughout the supply chain, that customized development of certain critical systems and components is necessary, then it should have agreement on standards and controls used in customized system and component development throughout the supply chain. Whether developed in-house or by a supply chain partner, additional scrutiny should be applied when custom development of systems and components occurs to ensure proper configuration to meet business and information security needs. Management should work with suppliers and partners to ensure that critical systems and components are identified. Effective management helps ensure that suppliers or the entity itself has a continued ability to maintain customized systems and components that are critical to the entity's operations. For example, having the source code, build scripts, and tests for a software component could enable an entity to have a third party maintain the system or component if necessary. For more information, refer to the “[Escrowed Source Code Agreements and Documentation](#)” section of this booklet.